

INGENIERÍA DE SOFTWARE I

Diagramas de Componentes UML

Estructura, modularidad y modelado de la arquitectura de ejecución física en sistemas de software modernos.



Conceptos de Modularidad

Entendiendo las bases de la arquitectura de componentes y la abstracción del diseño físico del software.

Definición y Propósito

¿Qué es el Diagrama?

Es un diagrama de estructura estática dentro de UML que muestra la organización y relación de dependencias entre los componentes de software en ejecución.

Abstrae los detalles del código fuente para centrarse en los módulos de software integrados y su arquitectura de entrega física.

Propósito en la Ingeniería

Permite visualizar el sistema tal como se implementará. Es crucial para definir los límites de desarrollo, modelar APIs y documentar la estructura para nuevos programadores.

Sirve como base fundamental para modelar la infraestructura física en el subsiguiente diagrama de despliegue.

Filosofía de Modularidad

Mantenibilidad Reemplazable

En la ingeniería basada en componentes, un módulo encapsula una funcionalidad específica y se comunica exclusivamente a través de interfaces explícitas.

Este enfoque reduce drásticamente el acoplamiento y eleva la cohesión del sistema, permitiendo un diseño modular e independiente de la infraestructura subyacente.

Al implementar este paradigma, cualquier componente puede sustituirse por otra versión mejorada (tipo "plug-and-play") siempre y cuando respete el contrato de sus interfaces.



Elementos Esenciales UML



Componente

Unidad modular reemplazable que encapsula el comportamiento interno. En UML se representa como un rectángulo con pestañas identificadoras.



Interfaz

Define un contrato de operaciones. Se divide en Proporcionada (lollipop), que expone servicios, y Requerida (socket), que solicita servicios ajenos.



Puerto y Conector

El puerto delimita las interacciones externas de forma segura. El conector une físicamente interfaces y guía los mensajes a través del sistema.

Interfaces como Contratos

En UML, las interfaces actúan como contratos estrictos entre los diferentes bloques de construcción de software.

La interfaz provista describe el comportamiento externo expuesto al entorno, garantizando la seguridad en el acceso a datos internos.

Al diseñar basándonos en contratos de interfaces provistas y requeridas, los desarrolladores pueden trabajar en paralelo con certeza absoluta de integración futura.



Clases vs. Componentes

Criterio	Diagrama de Clases	Diagrama de Componentes
Nivel de Abstracción	Conceptual y lógico (estructura interna del código).	Físico y de tiempo de ejecución (módulos reales).
Encapsulación	Detallada en métodos privados, públicos y protegidos.	Caja negra completa, expuesta solo por interfaces.
Reutilización	Mediante mecanismos de herencia y composición.	A través de la sustitución directa y modular.
Ejemplos Reales	Clase CarritoCompra , Clase Usuario	FacturacionAPI.dll , Frontend.war

Métricas de Cohesión

100%

Arquitectura Modular Desacoplada

Beneficios del Buen Diseño

Un diseño con alta cohesión y bajo acoplamiento maximiza la escalabilidad y minimiza el impacto ante cambios.

Podemos modelar matemáticamente la dependencia y estabilidad de un componente mediante la relación de acoplamiento eferente e aferente:

$$I = \frac{C_e}{C_a + C_e}$$

Donde I representa la inestabilidad del componente en función de sus conexiones.

Proceso de Creación

1. Identificar

Detectar los límites de ejecución: librerías, archivos compilados y fuentes de datos.

2. Definir

Esbozar las firmas de interfaces (servicios requeridos y provistos de cada elemento).

3. Conectar

Trazar dependencias lógicas y los ensamblajes de puertos e interfaces físicas.

4. Refinar

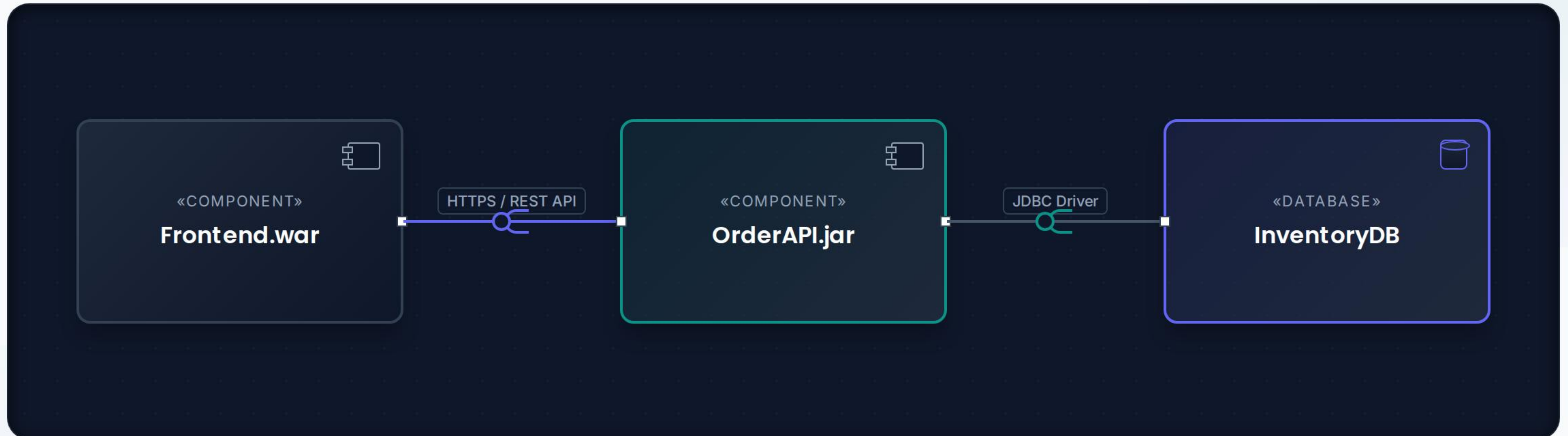
Verificar la coherencia del diseño físico con los diagramas de clases e infraestructura.



Caso Práctico Desarrollado

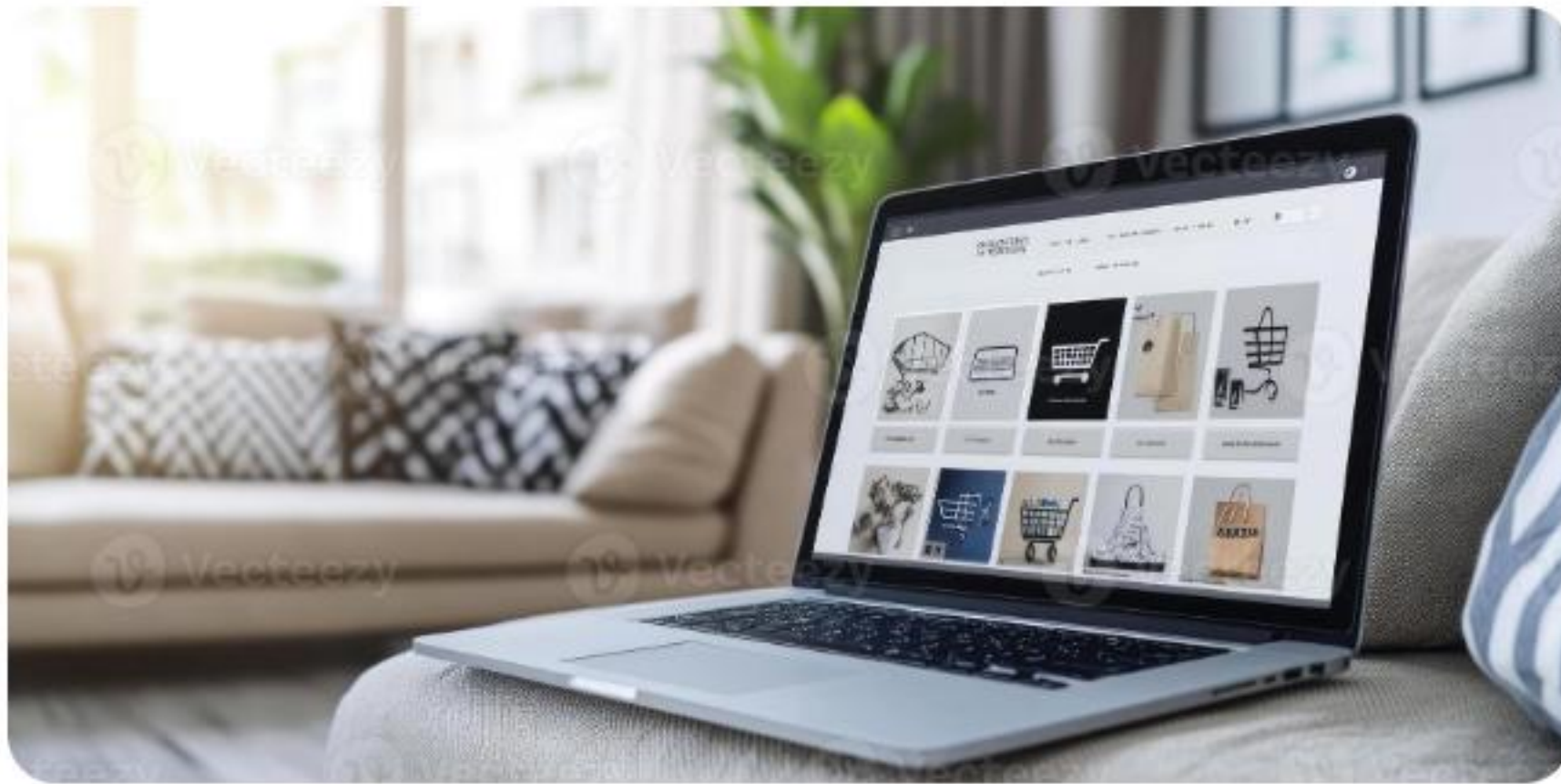
Estructuración física de componentes para una plataforma de Comercio Electrónico (E-Commerce) moderna.

Diagrama UML del Caso Práctico



Este diagrama de componentes ilustra la arquitectura de ejecución física de la aplicación. El cliente web (**Frontend.war**) delega peticiones a través de su puerto hacia la interfaz requerida que se acopla con la interfaz provista por el controlador de lógica de negocio (**OrderAPI.jar**). Éste, a su vez, mantiene un canal acoplado hacia la base de datos centralizada (**InventoryDB**) para consultar inventarios y transacciones.

Ejemplo: Tienda Online



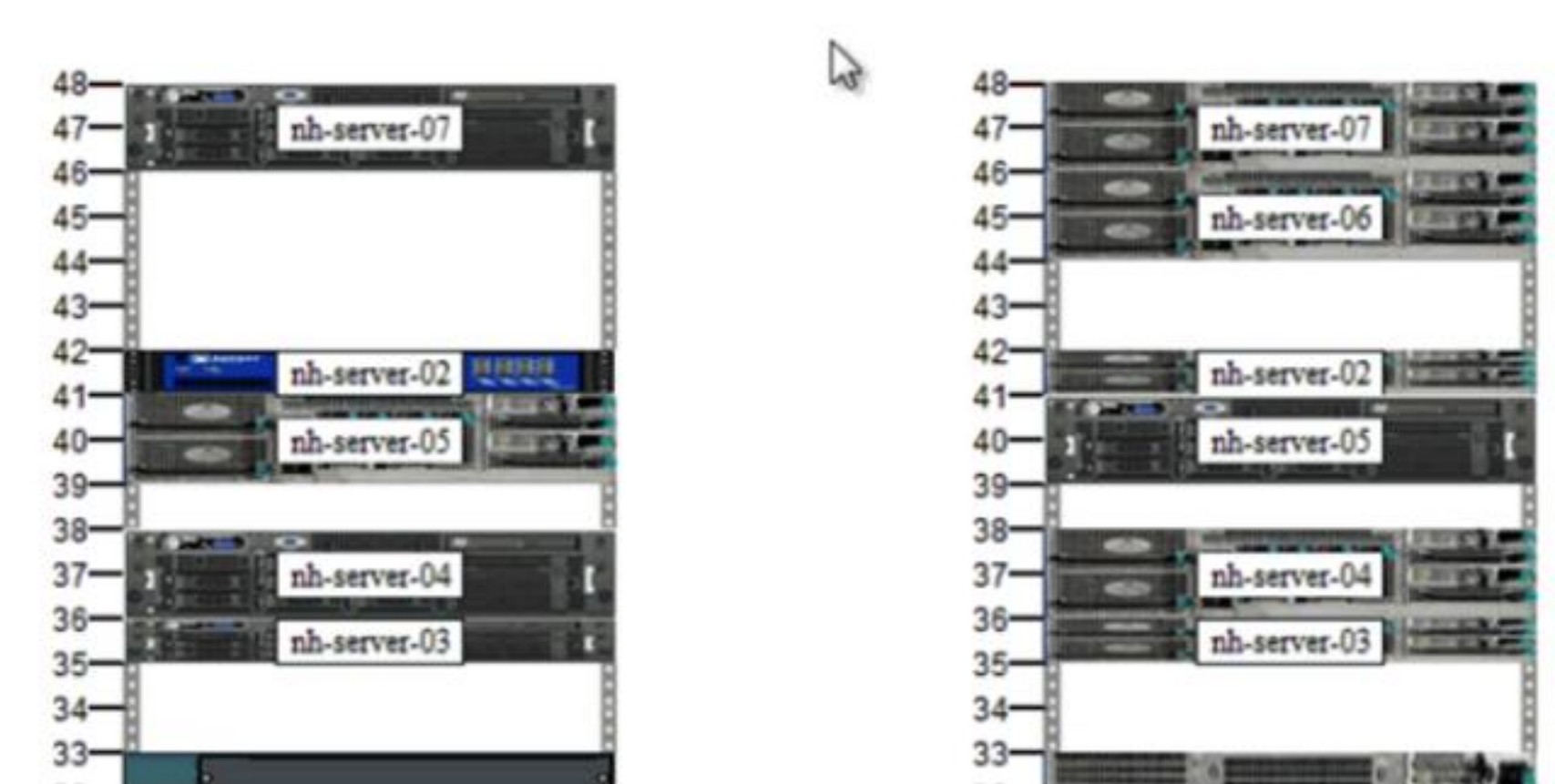
Cliente (Frontend.war)

Componente que interactúa con el usuario.
Consume la interfaz de backend mediante protocolos REST, requiriendo servicios de pago y pedidos.



Pedidos (OrderAPI.jar)

Componente de lógica de negocio. Provee interfaces para procesar compras y realizar cobros.
Requiere la conexión activa al motor de base de datos.



Datos (InventoryDB)

Componente de persistencia física. Expone interfaces exclusivas de consulta y actualización de inventario de forma segura.

¿Preguntas o Dudas?

Muchas gracias por su atención y tiempo.

Fundamentos de Ingeniería de Software | Presentación Académica

Image Sources



<https://i.ytimg.com/vi/Q9Qu-ddjDEE/maxresdefault.jpg>

Source: www.youtube.com



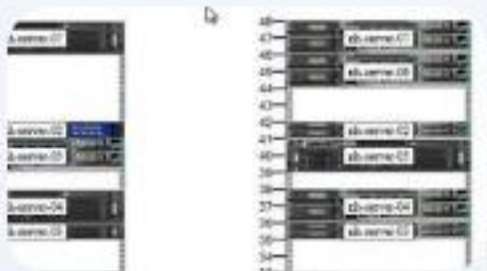
https://static.vecteezy.com/system/resources/previews/059/316/363/non_2x/wonderful-abstract-prefabricated-modular-housing-units-stacked-in-a-grid-minimalist-design-urban-setting-bright-daylight-architectural-visualization-original-png.png

Source: www.vecteezy.com



https://static.vecteezy.com/system/resources/previews/053/746/514/non_2x/modern-laptop-displaying-e-commerce-website-interface-photo.jpg

Source: www.vecteezy.com



https://www.device42.com/wp-content/uploads/2023/11/article-data-center-racks_Img0.png

Source: www.device42.com